Tarea 1 – Clasificador de Bayes
EL4106 - Semestre Otoño 2024

Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla

Fecha enunciado: 1 de Abril de 2024
Plazo entrega tarea: 14 de Abril de 2024

El objetivo de esta tarea es analizar, visualizar y clasificar datos usando las bibliotecas de software pandas, numpy, matplotlib y seaborn, las cuales serán utilizadas en el curso. El clasificador estadístico a implementar es el de Bayes. Se usará el conjunto de datos MAGIC Gamma Telescope Data Set, el cual corresponde a un conjunto de simulaciones de *air showers* generados por rayos gamma primarios v/s hadrones. El conjunto de datos forma parte del UC Irvine Machine Learning Repository. El conjunto de datos contiene 10 características y la clase, la cual puede ser h (hadrón) o g (no-hadrón). Hay 12332 ejemplos de rayos gamma y 6688 ejemplos de hadrones. El conjunto de datos contiene 11 columnas: las primeras 10 son las características y la última es la clase. Se entrega un archivo llamado magic04.data que contiene todas las muestras.

Se pide realizar lo siguiente:

Parte 1: Visualización y análisis de datos

1) Leer el dataset magic04.data usando la función pd.read_csv(). Se debe usar los siguientes nombres para las columnas:
   `['fLength', 'fWidth', 'fSize', 'fConc', 'fConc1', 'fAsym', 'fM3Long', 'fM3Trans', 'fAlpha', 'fDist', 'class']`
2) Separar los datos en dos subconjuntos, uno con los datos de la clase positiva (hadrón) y otro con los de la clase negativa (no-hadrón).
3) Para cada característica, generar histogramas para los datos de la clase negativa y los de la clase positiva. Para cada característica, ambos histogramas deben dibujarse usando un mismo gráfico. Para esto, se debe usar `plt.hist()` con `alpha=0.5`. Los histogramas deben estar normalizados. Elija un número de bins (particiones) que le parezca apropiado (investigue acerca de esto).
4) Calcular y graficar la matriz de correlación de las características, considerando el valor absoluto de las correlaciones. Incluir en este análisis la clase, como si fuera una característica extra (usar 0 para no-hadron o 1 para hadrón). Se debe usar como base la función de correlación existente en pandas. Para graficar se debe usar la función sns.heatmap( ) de seaborn.
5) Indicar por orden cuáles son las características más correlacionadas con la clase.
6) Indicar por orden cuáles son los 3 pares de características más correlacionadas entre sí. Averigüe sobre `unstack()` y `sort_values()`.
7) Graficar las dos características más correlacionadas en un *scatter plot* (gráfico de dispersión)
8) Indicar por orden cuáles son los 3 pares de características menos correlacionadas entre sí.
9) Graficar las dos características menos correlacionadas en un *scatter plot*. Compare el gráfico obtenido con el del punto 7.

Parte 2: Clasificación usando naive Bayes e histogramas

Se pide programar, entrenar y calibrar un clasificador naive Bayes para determinar en forma probabilística si un candidato corresponde a la clase positiva (hadrón) o a la negativa (no-hadrón). La regla de decisión del mencionado clasificador está dada por:

$$\frac{P(características|positiva)}{P(características|negativa)} \geq \Theta$$

El umbral Θ depende de los costos asociados a cada decisión ($c_{positiva}$ y $c_{negativa}$) y de las probabilidades a priori:

$$\Theta = \frac{c_{\text{positiva}} P(\text{negativa})}{c_{\text{negativa}} P(\text{positiva})},$$

En este caso concreto, las distribuciones de probabilidad condicional, i.e. las verosimilitudes, de ambos conjuntos no se tienen, por lo que deberá estimarlas utilizando histogramas normalizados.

10) Dividir la base de datos en 2 conjuntos representativos: entrenamiento (80%) y prueba (20%). Compruebe la representatividad de estos, verificando si la proporción de ambas clases en cada conjunto se mantiene cercana a la proporción del conjunto completo. Se recomienda usar la función `train_test_split()` para hacer la división, usando un `random_state` fijo.
11) Implemente un clasificador naive Bayes que use histogramas para representar las verosimilitudes.
12) Utilice la aproximación Naive Bayes y calcule los histogramas de cada clase a partir de las muestras del conjunto de entrenamiento. Elija un número de bins que le parezca apropiado, y use el mismo número de bins para todas las características. Se recomienda normalizar los histogramas.
13) Grafique los histogramas usando `plt.bar()`, con `alpha = 0.5`. En cada gráfico se debe superponer el histograma de una característica para la clase positiva y el histograma para la clase negativa.
14) Probar el clasificador: utilizando distintos valores de Θ, clasificar el conjunto de prueba y calcular la Tasa de Verdaderos Positivos y Tasa de Falsos Positivos, para luego generar la curva ROC (TVP vs TFP) para el conjunto de prueba. Recuerde que Θ puede tomar valores entre 0 e ∞, pero no todo el rango entrega información importante.
15) Genere la curva precisión-recall para el conjunto de prueba.

Parte 3: Clasificación usando gaussianas multivariantes

En esta sección se pide programar un clasificador basado en gaussianas multivariantes, el cual no usa la aproximación naive Bayes.

16) Entrenar un modelo gaussiano multidimensional para cada clase, calculando la media y covarianza, a partir del conjunto de entrenamiento. Puede basarse en las funciones `np.mean( )` y `np.cov( )` de numpy.
17) Implemente el clasificador.
18) Probar el clasificador: utilizando distintos valores de Θ, clasificar el conjunto de prueba y calcular la Tasa de Verdaderos Positivos y Tasa de Falsos Positivos para luego generar la curva ROC (TVP vs TFP) para el conjunto de prueba.
19) Genere la curva precisión-recall para el conjunto de prueba.

Parte 4: Comparación de resultados

20) Grafique las curvas ROC de los clasificadores entrenados en un único gráfico. Analice las métricas obtenidas por ambos clasificadores.
21) Grafique las curvas precisión-recall de los clasificadores entrenados en un único gráfico. Analice las métricas obtenidas por ambos clasificadores.
22) Compare los tiempos requeridos para entrenar ambos clasificadores

El código debe implementado en Python, usando *google colaboratory* (una herramienta que permite ejecutar notebooks de Python en la web: https://colab.research.google.com).

En la parte 1 se puede usar las librerías pandas, matplotlib y seaborn

En la parte 2 se puede usar las librerías scikit-learn (sólo para dividir los datos en entrenamiento y prueba), numpy para generar los histogramas, matplotlib para graficar los histogramas y curvas ROC y time.time() para medir los tiempos de entrenamiento.

En la parte 3 se puede usar sólo numpy para calcular las medias y covarianzas, matplotlib para graficar las curvas ROC y time.time() para medir los tiempos de entrenamiento.

En la parte 4 se puede usar matplotlib para graficar las curvas ROC en un mismo gráfico.

No se puede usar funciones que generen curvas ROC ni curvas precisión-recall, esto debe ser programado por los alumnos. Además, el alumno debe evitar divisiones por cero en los cálculos.

En esta tarea no se usará conjunto de validación, ya que no se explorarán distintos hiperparámetros, y se compararán sólo 2 clasificadores.

Los informes y códigos deben ser subidos a u-cursos a más tardar a las 23:59 del domingo 14 de abril. En el notebook entregado, incluir un breve comentario indicando cómo ejecutar su programa, o bien adjuntar un archivo README. El archivo del conjunto de datos a usar no puede ser renombrado ni modificado de ninguna forma. **Las Tareas atrasadas serán penalizadas con un punto de descuento por cada día extra de atraso, incluyendo fines de semana.**

Importante: La evaluación de la tarea considerará el correcto funcionamiento del programa, la inclusión de los resultados de los pasos pedidos en el informe, la calidad de los experimentos realizados y de su análisis, la inclusión de las partes importantes del código en el informe, así como la forma, prolijidad y calidad del mismo.

Nota: Se subirá un documento indicando la estructura del informe. Se recomienda leerlo cuidadosamente. El código se debe agregar como un anexo al final del informe en modo texto (no pantallazo).

Nota: Para detectar copias/plagios, se usará un software llamado turnitin. Los informes, con el código incluido como un anexo, se deben subir en el siguiente enlace:

https://uchile.turnitin.com/originality/hand-in-link?jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyZWR3b29kSWQiOiIzYzgxZGQ2Yi01OWE2LTQ2MGEtODE2OC01MGI0OWM1Y2YyYTEiLCJleHAiOjE3MTE3MTkzMDAsImlhdCI6MTcxMTYzMjkyNSwianRpIjoiN2RlYjcyNTQtYmU5Ny00YThkLTg0ODAtMDQ1MmFiYTljMTE0IiwidGVuYW50IjoidWNoaWxlIiwiZm9sZGVySWQiOiI2ZGJiNDEzNS0xOWRkLTRjN2MtODExMi01YjgwMmVkODczYmQifQ.gZUAQxEBpNIan2ymuEwEirnHrKCNRaqOUm5NiHg0q7E

Nota: Dado que turnitin no siempre acepta pdfs generados a partir de notebooks, los informes deben ser escritos en Word o latex.

Para evitar problemas, los alumnos deben programar todo lo que se pide por sí mismos y escribir el informe en su totalidad. No se puede reusar códigos de otros cursos o de Internet. Las copias/plagios serán penalizadas(os).